Window Lift System - High-Level Design

Document version 1.0

Status: released for review

1 Introduction

This High-Level Design (HLD) describes the software architecture for electric opening and closing of the front windows with one-touch operation and anti-pinch protection. It lists the software components, their ports and interfaces, the signals exchanged between them and the main functional flows.

The document is the reference for architects, developers, integrators and test engineers. Names of components, interfaces and signals are written exactly as they appear in the software architecture model.

1.1 Scope and assumptions

The design covers the application layer only. Basic software, bootloader and network management are described in separate documents. All components run in the same ECU and communicate through the runtime environment (RTE) using sender-receiver ports.

It is assumed that every signal has exactly one producer. Each signal is refreshed in the cycle time given in the signal list.

1.2 Terms

Term	Meaning
SWC	Software component, the smallest deployable unit of application software.
Port	Connection point of a component. Provided ports send data, required ports receive data.
Interface	Typed description of the data elements exchanged over a port.
Signal	Named data item with one producer and one or more consumers.
RTE	Runtime environment that connects the components.

2 Architecture Overview

The system consists of 4 software components. Each one has a clearly defined responsibility, and the dependencies between components are kept as small as possible. The following sections give the details of every component.

Components that provide data are placed close to the hardware abstraction, while controller components sit above them and make the decisions.

3 Software Components

3.1 Swc_WindowLiftCtrl

Main controller. It converts the switch input into a motor command, supports one-touch up and down, and stops the motor when a pinch event is reported.

Swc_WindowLiftCtrl has 3 port(s). The ports and the interfaces they use are listed in the table below.

Port	Direction	Interface
Rp_SwitchInput	required	If_WindowSwitch
Rp_Position	required	If_WindowPosition
Pp_MotorCmd	provided	If_MotorCmd

Dependencies: Swc_SwitchRx, Swc_PositionSensor.

Error handling: invalid or missing input data leads to a safe default value and the component sets its own status to invalid. The status is available to the diagnostic manager.

3.2 Swc_SwitchRx

Reads the window switch, debounces it with a 30 ms filter and classifies short and long presses.

Swc_SwitchRx has 1 port(s). The ports and the interfaces they use are listed in the table below.

Port	Direction	Interface
Pp_Switch	provided	If_WindowSwitch

Dependencies: none.

Error handling: invalid or missing input data leads to a safe default value and the component sets its own status to invalid. The status is available to the diagnostic manager.

3.3 Swc_PositionSensor

Counts Hall sensor pulses and calculates the window position in percent.

Swc_PositionSensor has 1 port(s). The ports and the interfaces they use are listed in the table below.

Port	Direction	Interface
Pp_Position	provided	If_WindowPosition

Dependencies: none.

Error handling: invalid or missing input data leads to a safe default value and the component sets its own status to invalid. The status is available to the diagnostic manager.

3.4 Swc_AntiPinchMon

Watches the motor current while the window closes and raises a pinch event when the current slope is above the calibrated limit.

Swc_AntiPinchMon has 2 port(s). The ports and the interfaces they use are listed in the table below.

Port	Direction	Interface
Rp_MotorCurrent	required	If_MotorCurrent
Pp_PinchEvent	provided	If_PinchEvent

Dependencies: Swc_WindowLiftCtrl.

Error handling: invalid or missing input data leads to a safe default value and the component sets its own status to invalid. The status is available to the diagnostic manager.

4 Interfaces

All interfaces are sender-receiver interfaces. The data elements of each interface are listed below.

Interface	Type	Data elements
If_WindowSwitch	Sender-Receiver	SwitchState, PressDuration
If_WindowPosition	Sender-Receiver	PositionPct, PositionValid
If_MotorCmd	Sender-Receiver	Direction, DutyCycle
If_PinchEvent	Sender-Receiver	PinchDetected, PinchPosition

5 Signal List

Producer and consumers of every signal are given in the table. A dash means that no consumer is defined.

Signal	Producer	Consumers	Cycle
Sig_WindowPosPct	Swc_PositionSensor	Swc_WindowLiftCtrl, Swc_AntiPinchMon	20 ms
Sig_PinchDetected	Swc_AntiPinchMon	Swc_WindowLiftCtrl	20 ms
Sig_WindowSwitchCmd	Swc_SwitchRx	Swc_WindowLiftCtrl	20 ms
Sig_MotorDutyCycle	Swc_WindowLiftCtrl	Swc_MotorDriverHw	20 ms

6 Functional Flows

6.1 One-touch close

Step 1: The driver presses the switch longer than 400 ms.

Step 2: Swc_SwitchRx reports a long press.

Step 3: Swc_WindowLiftCtrl closes the window until the end position or until Sig_PinchDetected is set.

7 Revision History

Version	Date	Change
1.0	2026-03-10	Initial release